Vulkan C# ECS Framework to Create a Generated Solar System

William Vickers Hastings

Games Academy, Falmouth University

Objectives

For my computing Artefact I aimed to create a solar system with generated planets around a spherical like object constructed for hexagons and pentagons. Additionally I wanted to use Vulkan and create an Entity Component System (ECS) for simulation and rendering:

- C# Vulkan Graphics Environment
- C# ECS Simulation Framework
- Terrain Generator with compute shaders
- Some Level of randomisation

Introduction

The ECS and compute shader workflow are the main focus here they are entirely of my own design, they became what made the project fun and pushed improvements to the terrain generation speed in the case of the compute shader. The ECS and transform system allows for hieratical transformations to mock up an animated solar system.

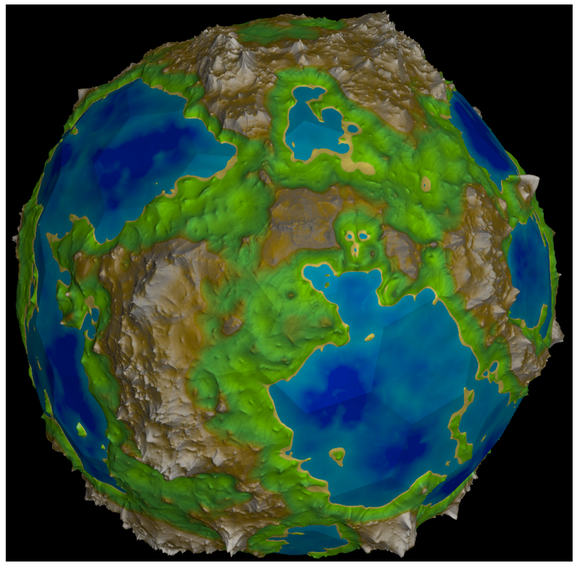

Figure 1:A planet generated by the arefact

Terrain Generation with Compute Shaders

The Terrain generator is based of Sebastian Lague's Procedural planets series[1], which uses layers of simplex noise[2] to generate mountains and flat regions. It has been modified to implement an terrain erosion approximation trick I found from Inigo Quilez [3] through a YouTube video [4]. The trick calculates the steepness pointing up hill from the current elevation point and raises or lowers the point depending on the steepness.

Most majorly the generator has been implemented as a vulkan compute shader which cuts generation time by 90%[5] on large meshes.

The compute shader workflow is has been abstracted into a class that takes a compute shader file, and some data. Which allowed easy implementation of multiple compute shaders.

The terrain generator at first ran on the vertex buffer, which then copied back to the CPU for Vertex normals. This copy & flush was so costly the CPU generator was faster[5]. I made a two stage compute shader for normals using atomic addition[5] which avoids the costly copy back & flush.

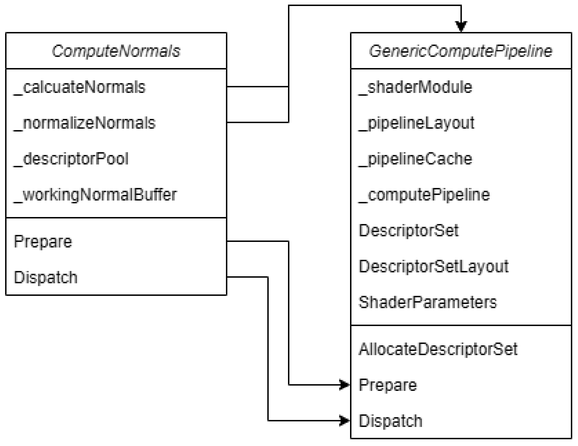

Figure 2:Structure of the two stage vertex normal compute shader. Two separate kernels run in separate generic compute pipelines using shared buffers

ECS Implementation

The ECS based off of my expirence with Unity's implementation. It is separated into Worlds which contain an entity manager to manager entities in that world. A world also contains systems that operate on the entities. The Entity manager uses dictionaries and hashsets for storage of entities, components and mapping between them. Components and entities have id numbers, Entities also have a version number allowing ids to be recycled. The entity id and version can be hashed to uniquely identify an entity. The component id number can be hashed with the entity hash to create a component signature id specific to that entity, which is used to load/store components in their instance dictionary.

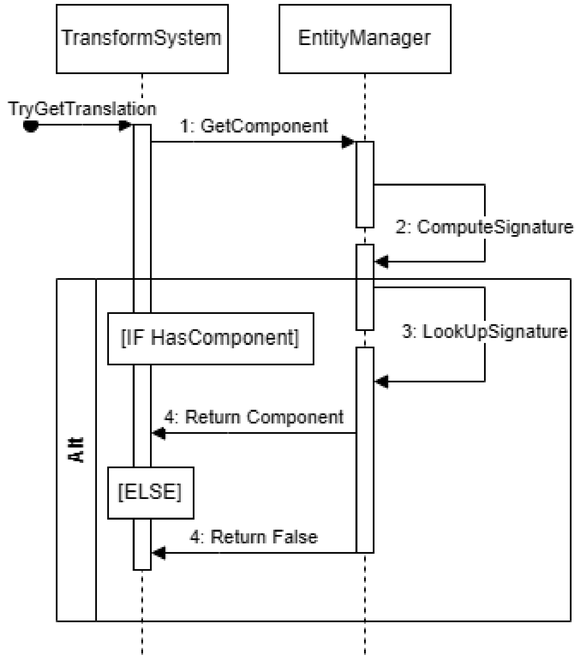

Figure 3:Sequence illustrating how a component is gotten for an entity

Conclusion

The ECS system and compute shader workflow are what is most interesting to me at the end. The planet generator I am unsatisfied with as its very time intensive to get configured correctly, but terrain about the hexa-sphere looks very pretty and unique. Runtime performance is entirely dependant on the amount of geometry in the world, which can be quite extreme with a lot of planets at high subdivisions. Further improvements to the I'd like to make, non-quadratic subdivision algorithm, mesh simplification, GPU driven rendering, shadow casting and other lighting improvements, actual orbital mechanics. Overall, terrain generation removed, this is the basis of a pretty solid ECS game engine that should probably be developed further.

Additional Information

Some extra features I didn't have the space to expand upon

- Triplanar mapping for texturing planets
- Subdividing the initial shape
- Point light rendering transparency
- Basic culling
- Unity like material workflow

References

[1] Sebastian Lague. [unity] procedural planets (e07: ocean depth). [Online]. Available: https://www.youtube.com/watch?v=OULxvDLojic

[2] K. Perlin, "Chapter 2: Noise hardware." [Online]. Available: https://userpages.cs.umbc.edu/olano/s2002c36/ch02.pdf

[3] I. Quilez. Inigo quilez. [Online]. Available: https://iquilezles.org/articles/morenoise/

[4] Josh's Channel. Better mountain generators that aren't perlin noise or erosion. [Online]. Available: https://www.youtube.com/watch?v=gsJHzBTPG0Y

[5] William Vickers Hastings. GA-undergrad-student-work-24-25/COMP305-2202796. [Online]. Available: https://github.falmouth.ac.uk/GA-Undergrad-Student-Work-24-25/COMP305-2202796